

Transformer Computation Walkthrough (Single-Head Self-Attention + FFN)

A fully worked numeric example with 3 tokens and 2 features

Goal of this document

This document explains the computations inside a **Transformer encoder block** using a tiny example. We show **every intermediate tensor** for:

- a single-head **self-attention** layer (including the attention matrix),
- the first **residual connection** (Add),
- a simple **feed-forward network (FFN)** applied token-wise,
- the second **residual connection** (Add).

We assume a batch size of $B = 1$, so we write tensors as $(T \times D)$ matrices.

Notation (shapes)

- T = number of tokens (sequence length). Here: $T = 3$.
- D = feature dimension per token (embedding size). Here: $D = 2$.
- Input token matrix: $X \in \mathbb{R}^{T \times D}$.
- Self-attention parameters: $W_Q, W_K, W_V \in \mathbb{R}^{D \times D}$.
- $Q, K, V \in \mathbb{R}^{T \times D}$ are the query/key/value matrices.
- Score matrix: $S = QK^\top \in \mathbb{R}^{T \times T}$.
- Attention weights: $A = \text{softmax}(S/\sqrt{D}) \in \mathbb{R}^{T \times T}$ (row-wise softmax).
- Attention output: $Y = AV \in \mathbb{R}^{T \times D}$.

Step 0: Input tokens X

We start with 3 tokens, each with 2 features:

$$X = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \end{bmatrix}$$

So:

$$x_1 = [1, 0], \quad x_2 = [0, 1], \quad x_3 = [1, 1].$$

Step 1: Build Q, K, V

Self-attention first projects tokens into three spaces:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Chosen weights (different from the previous example)

$$W_Q = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \end{bmatrix}, \quad W_V = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 2.0 \end{bmatrix}.$$

Computed projections

$$Q = \begin{bmatrix} 1.0 & 1.0 \\ 0.0 & 1.0 \\ 1.0 & 2.0 \end{bmatrix}, \quad K = \begin{bmatrix} 1.0 & 0.0 \\ 1.0 & 1.0 \\ 2.0 & 1.0 \end{bmatrix}, \quad V = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 2.0 \\ 1.0 & 2.0 \end{bmatrix}.$$

Interpretation (plain language):

- Each row of Q is “what this token is looking for” in other tokens.
- Each row of K is “what this token offers as an address/key”.
- Each row of V is “the information content this token can contribute”.

Step 2: Similarity scores $S = QK^\top$

We compute a score for every ordered pair of tokens:

$$S_{ij} = q_i \cdot k_j.$$

This produces a $(T \times T)$ matrix.

$$S = QK^\top = \begin{bmatrix} 1.0 & 2.0 & 3.0 \\ 0.0 & 1.0 & 1.0 \\ 1.0 & 3.0 & 4.0 \end{bmatrix}$$

Step 3: Scaling and softmax to get attention weights A

We scale scores by $\sqrt{D}$ (here $\sqrt{2}$) and apply **row-wise** softmax:

$$S' = \frac{S}{\sqrt{D}}, \quad A = \text{softmax}(S').$$

Scaled scores S'

$$S' \approx \begin{bmatrix} 0.707107 & 1.414214 & 2.121320 \\ 0.000000 & 0.707107 & 0.707107 \\ 0.707107 & 2.121320 & 2.828427 \end{bmatrix}$$

Attention matrix A (row-wise probabilities)

Each row sums to 1. Row i answers: “When updating token i , how much should I use each token j ?”

$$A \approx \begin{bmatrix} 0.140029 & 0.283995 & 0.575975 \\ 0.197776 & 0.401112 & 0.401112 \\ 0.074320 & 0.305695 & 0.619985 \end{bmatrix}$$

Important notes.

- A is generally **not symmetric**: token i attending to token j is not the same as j attending to i .
- The diagonal entries A_{ii} represent **self-attention**: a token is allowed to preserve its own information.

Step 4: Attention output $Y = AV$

Now we mix the value vectors using the attention weights:

$$y_i = \sum_{j=1}^T A_{ij} v_j \quad \implies \quad Y = AV.$$

$$Y \approx \begin{bmatrix} 0.716005 & 1.719942 \\ 0.598888 & 1.604448 \\ 0.694305 & 1.851361 \end{bmatrix}$$

Step 5: Residual connection #1 (Add)

A standard transformer encoder block uses a residual addition:

$$X_1 = X + Y.$$

This keeps the shape $(T \times D)$ unchanged.

$$X_1 \approx \begin{bmatrix} 1.716005 & 1.719942 \\ 0.598888 & 2.604448 \\ 1.694305 & 2.851361 \end{bmatrix}$$

Step 6: Feed-Forward Network (FFN) applied token-wise

After attention, each token is independently transformed by a small MLP (the same weights for every token):

$$\text{FFN}(x) = W_2 \phi(W_1 x + b_1) + b_2,$$

where ϕ is an activation function (we use ReLU).

Dimensions

We keep $D = 2$ at the block input/output, but we use an intermediate width $D_{ff} = 4$:

$$W_1 \in \mathbb{R}^{D \times D_{ff}} = \mathbb{R}^{2 \times 4}, \quad W_2 \in \mathbb{R}^{D_{ff} \times D} = \mathbb{R}^{4 \times 2}.$$

This is why people often write “ $D_{ff} = 4D$ ”: it is a **temporary expansion** inside the FFN.

Chosen FFN weights (also different from the previous example)

We set biases to zero to keep the arithmetic readable ($b_1 = \mathbf{0}$, $b_2 = \mathbf{0}$).

$$W_1 = \begin{bmatrix} 1.0 & -1.0 & 0.5 & 0.0 \\ 0.0 & 1.0 & 0.5 & -1.0 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \\ 1.0 & 1.0 \\ -0.5 & 0.5 \end{bmatrix}.$$

Compute $Z = X_1 W_1$ (**pre-activation, shape** $T \times D_{ff}$)

$$Z \approx \begin{bmatrix} 1.716005 & 0.003937 & 1.717973 & -1.719942 \\ 0.598888 & 2.005560 & 1.601668 & -2.604448 \\ 1.694305 & 1.157056 & 2.272833 & -2.851361 \end{bmatrix}$$

Apply **ReLU**: $A_1 = \max(0, Z)$

$$A_1 \approx \begin{bmatrix} 1.716005 & 0.003937 & 1.717973 & 0.000000 \\ 0.598888 & 2.005560 & 1.601668 & 0.000000 \\ 1.694305 & 1.157056 & 2.272833 & 0.000000 \end{bmatrix}$$

Project back to D : $F = A_1 W_2$ (**FFN output, shape** $T \times D$)

$$F \approx \begin{bmatrix} 3.433978 & 1.721910 \\ 2.200556 & 3.607229 \\ 3.967137 & 3.429889 \end{bmatrix}$$

Step 7: Residual connection #2 (Add)

The FFN output is added back:

$$X_2 = X_1 + F.$$

This is the **output of the transformer encoder block**. Shape is still $(T \times D)$.

$$X_2 \approx \begin{bmatrix} 5.149982 & 3.441851 \\ 2.799444 & 6.211677 \\ 5.661442 & 6.281249 \end{bmatrix}$$

Summary: what to remember

- Input and output of a transformer block keep the **same shape**: $(T \times D)$.
- Self-attention produces a **token-to-token** matrix $A \in \mathbb{R}^{T \times T}$ (the “who attends to whom” map).
- The FFN temporarily expands features (e.g., $D \rightarrow 4D \rightarrow D$) **per token**, without mixing tokens.
- Residual connections (Add) preserve information and make deep stacks trainable.